

Python 程序设计

第 6 讲 (a): Python 类 (class): 面向对象入门

HY Deng

hydeng@shiep.edu.cn



本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载（Operator Overloading）
- 3.3 向量合法性与异常（维数检查）

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

为什么要引入类 (Class)?

■ 之前我们已经学会:

- 用函数封装行为
- 用参数传递数据

■ 但现实问题中:

- 数据和行为天然绑定
- 状态需要长期保存

■ 类解决的问题:

“谁的数据？谁的方法？谁负责维护状态？”

灾难案例：只有函数的程序

- 同一类对象的数据散落在全局变量中
- 函数参数越来越多
- 状态容易出错、难以维护

```
x = 0 # x坐标  
y = 0 # y坐标  
vx = 1 # x方向速度  
vy = -1 # y方向速度  
def move(x, y, vx, vy):  
    x += vx  
    y += vy  
    return x, y
```

为什么这段程序很危险？

- 表面上：程序逻辑非常简单
- 实际上：同一个对象被拆成了 4 个变量
 - 位置： (x, y)
 - 速度： (v_x, v_y)
- 程序中没有任何地方明确声明：

“这些变量属于同一个对象”

状态是“谁的”？程序说不清楚

```
x = 0  
y = 0  
vx = 1  
vy = -1
```

■ 这 4 个变量：

- 属于谁？是否必须同时更新？

■ Python 并不知道：

x, y, v_x, v_y 是一个整体

接下来

■ 把:

x, y, v_x, v_y

■ 重新“合并”为一个对象

■ 让:

- 状态存在于对象内部
- 行为成为对象的方法

这一步，叫定义类：把“数据 + 操作 + 规则”
组织在一起

类的核心思想

- 类是一种抽象：
 - 描述“一类对象”的共同特征
 - 定义对象能做什么
- 类不是对象本身
- 类是对象的蓝图（**Blueprint**）

一个重要事实

- 到目前为止
- 我们几乎每天都在使用类
- 只是从未显式地写过 `class`

Python 中“万物皆对象”不是一句口号

Everything in Python is really an object.

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

例 1：字符串对象在维护什么状态？

```
str1 = "hello"  
str2 = str1.upper()  
print(str2)
```

■ 字符串对象内部保存：

- 字符序列
- 长度信息
- 编码规则

■ 调用 upper() 时：

- 不需要手动传入这些信息
- 对象自己知道

例 2：列表的 append 是谁在工作？

```
list1 = []  
list1.append('a')  
print(list1)
```

- append 并不是一个全局函数
- 它是 list 对象的方法

列表对象维护的状态

- 元素顺序
- 当前长度
- 内存空间管理
- 可变性规则

我们从不手动维护这些，因为：list 类已经替我们做了

例 3：字典的方法来自哪里？

```
dic = { 'name': 'SUEP',  
        'birth': 1951}  
for key, value in dic.items():  
    print(f'{key}: {value}')
```

- items() 是字典对象的方法
- 它知道：
 - 键
 - 值
 - 内部存储结构

三个例子的一致结构

■ 字符串、列表、字典都满足：

- 有内部状态
- 有专属方法
- 状态不会随意暴露

对象．方法（）

这就是类在工作样子

对比“只有函数的程序”

■ 那个程序中：

- 没有对象
- 没有统一状态
- 没有方法归属

■ 所有规则靠人为约定

这正是类要补上的部分

现在，只差最后一步

- 我们已经：
 - 熟练使用内置类
- 接下来要做的：
 - 为自己的问题
 - 设计自己的类

写 class 不是新东西，而是“同样的事自己做一遍”

下一步

- 把之前的“只有函数的程序”
- 改写为：

`object.method()`

- 明确：
 - 状态在哪
 - 谁能改

这一步，才是真正的“面向对象”

什么是对象？

- 对象 = 一组数据 + 一组操作这些数据的方法
- 对象 = 有状态 (**state**) 的实体
- 核心问题不是语法，而是三件事：
 - 数据是谁的？
 - 谁负责维护这些数据的一致性？
 - 谁可以直接访问 / 修改？

面向对象的直觉

把“数据”和“操作数据的函数”绑在一起

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

现实中的学生对象

一个学生通常有什么？

■ 属性（数据）

- 姓名
- 学号
- 年龄

■ 行为（方法）

- 自我介绍
- 过生日（年龄 +1）

核心理念

类 = 对“同一类对象”的抽象描述

什么是类 (Class)?

■ 类是:

- 一类对象的抽象描述
- 对象的模板

■ 类描述:

- 对象拥有什么数据 (属性)
- 对象能做什么事情 (方法)

类 $\longrightarrow$ 对象

定义一个最简单的 Student 类

```
class Student:  
    pass
```

- class: 定义类的关键字
- Student: 类名
- 冒号 (:): 之后是类体

类名应该如何命名？

■ 类名使用：

- 大驼峰命名法

■ 正确示例：

- Student
- CourseRecord

■ 不推荐：

- student
- student_class

学生类的属性和方法

```
class Student:
    def __init__(self, name, sid, age):
        self.name = name
        self.sid = sid
        self.age = age

    def introduce(self):
        print(f"My name is {self.name}, id={self.sid}")

    def birthday(self):
        self.age += 1
```

什么是属性（attribute）

- 属性是：
 - 绑定在对象上的变量
 - 描述对象“当前状态”
- 不同对象有不同属性值

重要区分

- 1 **self.name** 是对象的属性
- 2 **name** 只是一个普通局部变量

由类创建对象

```
s1 = Student()  
s2 = Student()
```

- Student 是类
- s1, s2 是对象
- 它们:
 - 类型相同
 - 但不是同一个对象

类和对象的关系

```
s1 = Student("Alice", "202301", 18)
s2 = Student("Bob", "202302", 19)
```

- Student: 类（模板、蓝图）
- s1, s2: 对象（实例）
- 不同对象:
 - 共享同一套方法
 - 各自拥有独立的数据

__init__ 的真正作用

- `__init__` 在对象创建之后自动调用
- 它的职责只有一个：
 - 初始化对象的状态（属性）

一句话总结

`__init__` = 给“新对象”装配初始数据

self 的本质

- `self` = 当前对象本身
- 它不是关键字，只是一个约定俗成的名字
- 每个实例方法的第一个参数，自动传入

关键理解

`self.name` = “这个对象的 `name`”

方法调用的真实展开

```
s1.introduce()
```

等价于:

```
Student.introduce(s1)
```

- s1 自动传给 self
- 方法才能知道:
 - 自己在操作 “哪一个对象”

方法调用时发生了什么？

```
s1 = Student("Alice", "2023001")
```

- 实际执行的是：

```
Student.__init__(s1, "Alice", "2023001")
```

- self 自动绑定为 s1

给学生对象添加行为

```
class Student:
    def __init__(self, name, sid):
        self.name = name
        self.id = sid

    def introduce(self):
        print(f"My name is {self.name}, id={self.
            sid}")
```

- 方法 = 属于对象的函数
- 第一个参数必须是 self

对象如何调用方法

```
s = Student("Alice", "2023001")  
s.introduce()
```

■ s.introduce()

■ 等价于:

Student.introduce(s)

方法 vs 普通函数

■ 普通函数:

- 数据靠参数传递
- 不记住状态

■ 类的方法:

- 直接操作对象内部属性
- 负责维护对象的一致性

面向对象的本質

谁的数据，谁负责修改

核心总结

- 类：对象的模板
- 对象：有状态、有行为
- 属性：对象保存的数据
- 方法：操作对象状态的函数
- self：对象自身
- `__init__`：对象的出生规则

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

一个普通的类

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

■ name, age 是什么?

■ name, age 是对象的状态 (属性)

一个普通的类

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

- name, age 是什么?
- name, age 是对象的状态 (属性)

直接访问属性的问题

- 看起来很自然:

`s.age = -3`

- 但这合理吗?
- 类有没有机会阻止?

没有任何防护

用方法修改属性

```
class Student:
    def set_age(self, age):
        if age < 0:
            raise ValueError("age must be
                               positive")
        self.age = age
```

- 合法性检查
- 状态由类控制

接口的“不自然感”

- 用户要记住:

```
s.set_age(20)
```

- 而不是直觉上的:

```
s.age = 20
```

这在 Python 中被称为“不够优雅”

@property 解决了什么？

- 对外：仍然是属性访问
- 对内：仍然是方法逻辑

s.age = 20, 看起来像赋值



实际执行的是方法

property 最直观的特点

- 普通方法调用: `obj.method()`
- property 调用: `obj.attr`

没有 `()`，看起来就是一个属性

property 最直观的特点

- 普通方法调用: `obj.method()`
- property 调用: `obj.attr`

没有 `()`, 看起来就是一个属性

property 的完整形式

```
class Student:
    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("age must be >= 0")
        self._age = value
```

property 的使用方式

```
s = Student(18)
s.age = 20
print(s.age)
```

- 但每一步都有检查
- 状态完全由类掌控

property 的本质

- 对用户：
 - 像属性
- 对类作者：
 - 是函数
 - 可以写逻辑、检查、计算

这是 Python 非常 “Pythonic” 的设计

方法 vs property

```
class Circle:
    def area_method(self):
        return 3.14 * self.r ** 2
    @property
    def area(self):
        return 3.14 * self.r ** 2
```

■ `c.area_method()`

■ `c.area`

第二种更符合“面积是一个量”的直觉

getter 的角色

■ 当你写:

`x = obj.attr`

■ 实际调用的是:

`obj.attr()` 的 getter 方法

但你不需要写 ()

setter 的角色

■ 当你写:

`obj.attr = value`

■ 实际执行的是:

`attr` 的 `setter` 方法

类获得了“拦截赋值”的能力

getter 与 setter 各做什么？

■ getter:

- 返回值
- 计算派生量

■ setter:

- 合法性检查
- 维护不变量

这是一种“职责清晰”的设计

只读属性（Read-only property）：没有 setter 会怎样？

- 只有 getter
- 没有 setter

`obj.attr` 可以

`obj.attr = value` 报错

属性存在，但不能被修改

什么时候应该是只读属性？

■ 派生量：

- 面积
- 模长
- 维数

■ 不希望用户直接改的状态

“算出来的量，不应该被赋值”

只读属性

```
class Circle:
    def __init__(self, r):
        self._r = r

    @property
    def area(self):
        return 3.14159 * self._r ** 2
```

- 只有 getter
- 没有 setter

property 的真正价值

- 对外:
 - 简洁
 - 符合直觉
- 对内:
 - 可控
 - 可维护

接口稳定，内部可随时重构

一句话总结 @property

- @property 是：
 - 用函数的方式
 - 提供属性的接口
- 调用函数或方法时不用加 ()
- 可以控制读写

“像属性一样用，像函数一样安全”

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

数学视角下的问题

- 向量是一个整体对象
- 坐标只是它的内部表示
- 运算应该属于向量本身

Vector 类的数学抽象

- 一个向量应包含:
 - 分量 (components)
 - 维数 (dimension)
- 一个向量应支持:
 - 加法
 - 内积
 - 范数

Vector 类的数学抽象

- 一个向量应包含:
 - 分量 (components)
 - 维数 (dimension)
- 一个向量应支持:
 - 加法
 - 内积
 - 范数

定义 Vector 类 (骨架)

```
class Vector:  
    pass
```

- 这是一个“数学概念占位符”
- 后续逐步为它添加结构与规则

__init__: 定义向量的分量

```
class Vector:
    def __init__(self, components):
        self.components = components
```


创建向量对象

```
v1 = Vector([1, 2])  
v2 = Vector([3, 4])
```

- Vector 是抽象
- v1, v2 是具体向量

向量加法作为方法

```
class Vector:
    def __init__(self, components):
        self.components = components

    def add(self, other):
        result = []
        for a, b in zip(self.components, other
                        .components):
            result.append(a + b)
        return Vector(result)
```

self 的数学含义

- self 就是“当前向量”
- self.add(other) 对应:

$$\mathbf{v} + \mathbf{w}$$

定义向量的二范数

```
import math

class Vector:
    def __init__(self, components):
        self.components = components

    def norm(self):
        return math.sqrt(sum(x**2 for x in
                               self.components))
```

数学对应关系

$$\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}$$

- 代码结构 数学公式结构
- 类让“公式活了起来”

Vector 对象的本质

- 一个 Vector 对象代表一个具体向量
- 分量是内部状态
- 运算是对于状态的映射

Vector = components + operations

为什么这是“工程级写法”？

- 向量逻辑集中在一个地方
- 易于扩展 (内积、投影、归一化)
- 接近 NumPy / SciPy 的设计思想

你正在“手写一个小型线性代数库”

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

从“能算”到“像数学一样算”

■ 目前我们已经可以:

- `v.add(w)`

- `v.norm()`

■ 但数学中更自然的是:

$$\mathbf{v} + \mathbf{w}, \quad \|\mathbf{v}\|$$

■ 运算符重载的目标:

让代码的语法形态贴近数学公式

运算符的本质

- Python 中运算符并非“特殊语法”
- 本质是对特定方法的调用

$$v + w \iff v.__add__(w)$$
$$\text{len}(v) \iff v.__len__()$$

重载向量加法

```
class Vector:
    def __init__(self, components):
        self.components = components

    def __add__(self, other):
        result = []
        for a, b in zip(self.components, other
                        .components):
            result.append(a + b)
        return Vector(result)
```

使用效果

```
v = Vector([1, 2])
```

```
w = Vector([3, 4])
```

```
u = v + w
```

$$(1, 2) + (3, 4) = (4, 6)$$

运算符重载的第一原则

- 运算符含义必须符合数学直觉
- + 应该满足:
 - 封闭性
 - 交换性 (如适用)
 - 结合性 (如适用)
- 否则代码将“误导使用者”

重载向量减法

```
def __sub__(self, other):  
    result = []  
    for a, b in zip(self.components, other  
                    .components):  
        result.append(a - b)  
    return Vector(result)
```

向量的数乘

```
def __mul__(self, scalar):  
    result = [scalar * x for x in self.  
               components]  
    return Vector(result)
```

数学解释

$$\alpha \mathbf{v} = (\alpha v_1, \alpha v_2, \dots)$$

- 此处约定: * 表示标量乘
- 点积不应滥用 *

点积为什么不推荐用 *

- 数学中 $\mathbf{v} \cdot \mathbf{w}$ 有多种约定
- Python 中 * 容易产生歧义
- NumPy 中:
 - *: 逐元素
 - @: 矩阵/向量乘法

使用 @ 表示点积

```
def __matmul__(self, other):  
    return sum(a * b for a, b in zip(self.  
        components, other.components))
```

使用效果

```
v = Vector([1, 2])  
w = Vector([3, 4])  
  
dot = v @ w
```

$$\mathbf{v} \cdot \mathbf{w} = 11$$

重载 len()

```
def __len__(self):  
    return len(self.components)
```

数学意义

$$\text{len}(\mathbf{v}) = \text{dim}(\mathbf{v})$$

- 代码表达数学结构
- 不是“技巧”，是设计

向量的打印形式

```
def __repr__(self):  
    return f"Vector({self.components})"
```

认知跃迁总结

■ 运算符重载让代码:

- 像数学
- 像论文
- 像工具

■ 这是数值计算 / 科学计算库的基础

你已经站在 NumPy 的设计起点上

本讲的主要内容

1 1. 类的引入

- 1.1 为什么要引入类
- 1.2 回顾：已使用的类

2 2. 类的基本概念

- 2.1 从学生这个对象说起
- 2.2 @property, setter, getter

3 3. 类的提升

- 3.1 向量 Vector 类示例
- 3.2 运算符重载 (Operator Overloading)
- 3.3 向量合法性与异常 (维数检查)

数学中的“非法运算”

- 数学中并非所有运算都有定义

$$\mathbb{R}^2 + \mathbb{R}^3 \text{ 无意义}$$

- 数学上我们会:
 - 明确写出“无定义”
 - 或直接拒绝该运算
- 程序中也必须如此

一个危险的例子

```
v = Vector([1, 2])  
w = Vector([3, 4, 5])  
  
u = v + w    # 会发生什么?
```

为什么这比报错更糟？

- 没有异常 $\neq$ 正确
- 这是悄悄产生错误结果
- 在数值计算中：
 - 错误会被不断放大
 - 结果看起来“很合理”

数值计算最怕：silent bug

合法性检查的设计思想

■ 数学中:

未定义 $\Rightarrow$ 禁止运算

■ 程序中:

非法输入 $\Rightarrow$ 抛出异常

■ 这不是“防御式编程”

■ 而是数学结构的显式表达

异常 错误

■ 异常是:

- 一种被设计出来的“中断机制”
- 用来阻止错误传播

■ 数学类库中:

- 报错是责任
- 不报错是失职

统一的私有检查函数

```
class Vector:
    def __init__(self, components):
        self.components = components

    def _check_dim(self, other):
        if len(self.components) != len(other.
            components):
            raise ValueError("Vector
                dimensions do not match")
```

安全的向量加法

```
def __add__(self, other):  
    self._check_dim(other)  
    return Vector([  
        a + b for a, b in zip(self.  
            components, other.components)  
    ])
```

一致性非常重要

```
def __sub__(self, other):  
    self._check_dim(other)  
    return Vector([  
        a - b for a, b in zip(self.  
            components, other.components)  
    ])  
  
def __matmul__(self, other):  
    self._check_dim(other)  
    return sum(a * b for a, b in zip(self.  
        components, other.components))
```


异常信息也是数学的一部分

- 错误信息应回答:
 - 哪里错?
 - 为什么错?
- 好的异常信息 好的数学注释

"Vector dimensions do not match"

不仅要维数对，还要“对象对”

- 数学中不会拿向量去加标量
- 程序中也应明确禁止

真正目标

- 数学对象 任意数据
- 运算定义域是结构的一部分
- 类 + 异常 = 数学公理的程序化

你不是在写代码，而是在写数学系统

当前 Vector 类

```
class Vector:
    def __init__(self, components):
        self.components = components

    def _check_vector(self, other):
        if not isinstance(other, Vector):
            raise TypeError("Operand must be a Vector")

    def _check_dim(self, other):
        if len(self.components) != len(other.components):
            raise ValueError("Vector dimensions do not match")

    def __add__(self, other):
        self._check_vector(other)
        self._check_dim(other)
        return Vector([a + b for a, b in zip(self.components,
                                                other.components)])
```

当前 Vector 类

接上一页

```
def __sub__(self, other):  
    self._check_vector(other)  
    self._check_dim(other)  
    return Vector([a - b for a, b in zip(self.components,  
                                           other.components)])  
  
def __matmul__(self, other):  
    self._check_vector(other)  
    self._check_dim(other)  
    return sum(a * b for a, b in zip(self.components, other.  
                                       components))
```

- 1. 类的引入
- 2. 类的基本概念
- 3. 类的提升

3.1 向量 Vector 类示例

3.2 运算符重载 (Operator Overloading)

3.3 向量合法性与异常 (维数检查)

Thanks!